


```

)
import matplotlib.pyplot as plt

import torch
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    DataCollatorWithPadding,
)
from datasets import Dataset
import warnings
warnings.filterwarnings("ignore")

print("✅ Imports complete")
print(f"PyTorch version: {torch.__version__}")
print(f"GPU available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")

```

```

✅ Imports complete
PyTorch version: 2.11.0+cu128
GPU available: True
GPU: Tesla T4

```

✓ Section 1: Load Your Dataset

Upload your labeled CSV and define your label map.

Your CSV must have at least two columns: `text` (the post/comment) and `label` (your string label).

```

# — TODO —
# Define YOUR label map below.
# Keys are the string labels in your CSV; values are integers starting at 0.
# Add or remove entries to match your actual labels (2-4 labels supported).
#
# The example below is ILLUSTRATIVE ONLY (the r/nba taxonomy from the project
# page). DELETE it and use your own community's labels – submitting the
# example unchanged will not pass.
#
LABEL_MAP = {

```

```

"analysis": 0, # ← Replace with your first label
"hot_take": 1, # ← Replace with your second label
"anecdote": 2, # ← Replace with your third label (remove if you have 2 labels)
# "fourth_label": 3, # ← Uncomment if you have a fourth label
}

```

```
# — END TODO —————
```

```

ID_TO_LABEL = {v: k for k, v in LABEL_MAP.items()}
NUM_LABELS = len(LABEL_MAP)
print(f"Labels: {LABEL_MAP}")
print(f"Number of labels: {NUM_LABELS}")

```

```

Labels: {'analysis': 0, 'hot_take': 1, 'anecdote': 2}
Number of labels: 3

```

```

# Upload your CSV from your computer
from google.colab import files
print("Select your labeled dataset CSV file...")
uploaded = files.upload()
CSV_PATH = list(uploaded.keys())[0]
print(f"Uploaded: {CSV_PATH}")

```

Select your labeled dataset CSV file...

dataset.csv

dataset.csv(text/csv) - 92134 bytes, last modified: 6/23/2026 - 100% done
 Saving dataset.csv to dataset.csv
 Uploaded: dataset.csv

```

# Load and validate your dataset
df = pd.read_csv(CSV_PATH)

```

```

# — TODO (if needed) —————
df.columns = [c.lower().strip() for c in df.columns]
# Example: df = df.rename(columns={"post": "text", "category": "label"})
# — END TODO —————

```

```

print(f"Columns: {df.columns.tolist()}")
print(f"Total examples: {len(df)}")
print()
print("Label distribution:")
print(df["label"].value_counts())

```

```

# Validate all labels are in LABEL_MAP
unknown = set(df["label"].unique()) - set(LABEL_MAP.keys())

```

```

if unknown:
    print(f"\n🚩 Labels in CSV not found in LABEL_MAP: {unknown}")
    print("Update your LABEL_MAP above to include all labels.")
else:
    print("\n✅ All labels match your LABEL_MAP")

```

```

# Convert string labels to integers
df["label_id"] = df["label"].map(LABEL_MAP)
df = df.dropna(subset=["label_id"])
df["label_id"] = df["label_id"].astype(int)

```

```

Columns: ['text', 'label', 'notes']
Total examples: 200

```

```

Label distribution:
label
analysis    90
hot_take    72
anecdote    38
Name: count, dtype: int64

```

```

✅ All labels match your LABEL_MAP

```

✓ Section 2: Prepare Data for Training

Splits your dataset into train / validation / test sets and tokenizes the text.

```

# Train / val / test split – 70% / 15% / 15%
# Stratified so each split has roughly the same label distribution.
train_df, temp_df = train_test_split(
    df, test_size=0.30, random_state=42, stratify=df["label_id"]
)
val_df, test_df = train_test_split(
    temp_df, test_size=0.50, random_state=42, stratify=temp_df["label_id"]
)

print(f"Train: {len(train_df)} examples")
print(f"Validation: {len(val_df)} examples")
print(f"Test: {len(test_df)} examples")
print()
print("Train label distribution:")
print(train_df["label"].value_counts())
print()

```

```
print("Test label distribution:")
print(test_df["label"].value_counts())

# Reset indices (needed for clean HuggingFace Dataset conversion)
train_df = train_df.reset_index(drop=True)
val_df   = val_df.reset_index(drop=True)
test_df  = test_df.reset_index(drop=True)
```

```
Train: 140 examples
Validation: 30 examples
Test: 30 examples

Train label distribution:
label
analysis    63
hot_take    50
anecdote    27
Name: count, dtype: int64

Test label distribution:
label
analysis    14
hot_take    11
anecdote     5
Name: count, dtype: int64
```

```
# Load tokenizer and tokenize all splits
MODEL_NAME = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tokenize(examples):
    return tokenizer(examples["text"], truncation=True, max_length=256)

def make_dataset(df_split):
    ds = Dataset.from_pandas(
        df_split[["text", "label_id"]].rename(columns={"label_id": "labels"})
    )
    return ds.map(tokenize, batched=True)

train_dataset = make_dataset(train_df)
val_dataset   = make_dataset(val_df)
test_dataset  = make_dataset(test_df)

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
print("✅ Tokenization complete")
print(f"Sample keys: {list(train_dataset[0].keys())}")
```

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads
config.json: 100%                               483/483 [00:00<00:00, 53.6kB/s]

tokenizer_config.json: 100%                     48.0/48.0 [00:00<00:00, 4.91kB/s]

vocab.txt: 100%                                232k/232k [00:00<00:00, 949kB/s]

tokenizer.json: 100%                           466k/466k [00:00<00:00, 3.21MB/s]

Map: 100%                                      140/140 [00:00<00:00, 1642.19 examples/s]

Map: 100%                                      30/30 [00:00<00:00, 1009.28 examples/s]

Map: 100%                                      30/30 [00:00<00:00, 914.54 examples/s]

✅ Tokenization complete
Sample keys: ['text', 'labels', 'input_ids', 'token_type_ids', 'attention_mask']

```

✓ Section 3: Fine-Tune Your Model

Loads `distilbert-base-uncased` with a classification head and fine-tunes it on your training data.

Training runs for 3 epochs and takes **5–15 minutes** on a T4 GPU.

Hyperparameter note: The defaults below work well for datasets of 100–500 examples.

If you change any values, note what you changed and why in your README.

```

# Load DistilBERT with a classification head
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=NUM_LABELS,
    id2label=ID_TO_LABEL,
    label2id=LABEL_MAP,
)
print(f"✅ Model loaded: {MODEL_NAME}")
print(f"Output labels: {NUM_LABELS}")

```

```

model.safetensors: 100%                               268M/268M [00:02<00:00, 2.23GB/s]

Loading weights: 100%                               100/100 [00:00<00:00, 3763.29it/s]

[transformers] DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key | Status |
-----+-----+
vocab_layer_norm.bias | UNEXPECTED |
vocab_projector.bias | UNEXPECTED |
vocab_transform.bias | UNEXPECTED |
vocab_transform.weight | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
classifier.weight | MISSING |
classifier.bias | MISSING |
pre_classifier.weight | MISSING |
pre_classifier.bias | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
✅ Model loaded: distilbert-base-uncased
Output labels: 3

```

```

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return {"accuracy": accuracy_score(labels, predictions)}

```

```

# — Hyperparameters —————
# num_train_epochs — passes through the training data; 3 is a good default
#                   — for small datasets. Increase cautiously; more epochs
#                   — risk overfitting on 200 examples.
# learning_rate     — 2e-5 is the standard starting point for fine-tuning
#                   — BERT-family models. Lower → slower but more stable.
# per_device_train_batch_size — 16 fits T4 GPU comfortably.
#                   — Reduce to 8 if you get out-of-memory errors.
# —————

training_args = TrainingArguments(
    output_dir="./takemeter-model",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    learning_rate=2e-5,
    weight_decay=0.01,
    warmup_steps=50,
    eval_strategy="epoch",

```

```

    save_strategy="epoch",
    save_total_limit=1,
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
    logging_steps=10,
    report_to="none",
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)

print("Starting fine-tuning... (5-15 minutes on T4 GPU)")
trainer.train()
print("\n✅ Fine-tuning complete")

```

Starting fine-tuning... (5-15 minutes on T4 GPU)

 [27/27 00:21, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Accuracy
1	No log	1.096843	0.233333
2	1.101776	1.062270	0.566667
3	1.075791	1.011842	0.633333

Writing model shards: 100%

1/1 [00:00<00:00, 1.68it/s]

Writing model shards: 100%

1/1 [00:00<00:00, 1.79it/s]

Writing model shards: 100%

1/1 [00:03<00:00, 3.22s/it]

✅ Fine-tuning complete

✓ Section 4: Evaluate Fine-Tuned Model on Test Set

Runs inference on your locked test set and generates metrics and a confusion matrix.
These numbers go directly into your evaluation report.


```

# Run inference on the test set
print("Running inference on test set...")
ft_output = trainer.predict(test_dataset)
ft_pred_ids = np.argmax(ft_output.predictions, axis=-1)
ft_true_ids = ft_output.label_ids

ft_probs = torch.nn.functional.softmax(
    torch.tensor(ft_output.predictions), dim=-1
).numpy()

# Overall accuracy
ft_accuracy = accuracy_score(ft_true_ids, ft_pred_ids)
print(f"\n🎯 Fine-tuned model accuracy: {ft_accuracy:.3f}")

# Per-class metrics
label_names = [ID_TO_LABEL[i] for i in range(NUM_LABELS)]
print("\nPer-class metrics (fine-tuned model):")
print(classification_report(ft_true_ids, ft_pred_ids, target_names=label_names, zero_division=0))

```

Running inference on test set...

🎯 Fine-tuned model accuracy: 0.433

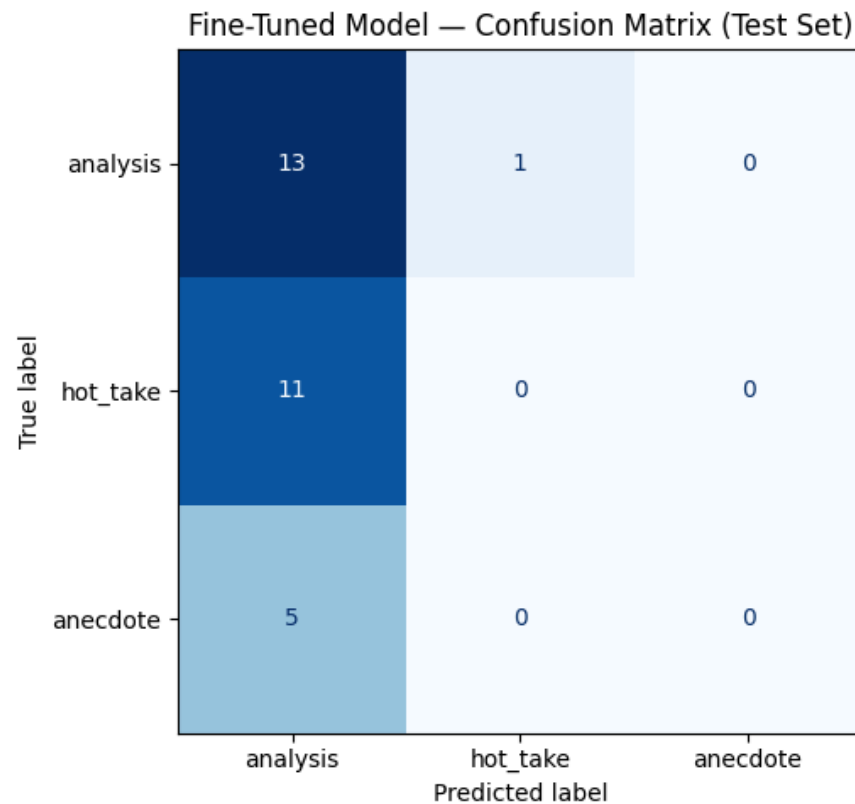
Per-class metrics (fine-tuned model):

	precision	recall	f1-score	support
analysis	0.45	0.93	0.60	14
hot_take	0.00	0.00	0.00	11
anecdote	0.00	0.00	0.00	5
accuracy			0.43	30
macro avg	0.15	0.31	0.20	30
weighted avg	0.21	0.43	0.28	30

```

# Confusion matrix
cm = confusion_matrix(ft_true_ids, ft_pred_ids)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=label_names)
fig, ax = plt.subplots(figsize=(7, 5))
disp.plot(ax=ax, cmap="Blues", colorbar=False)
ax.set_title("Fine-Tuned Model - Confusion Matrix (Test Set)")
plt.tight_layout()
plt.savefig("confusion_matrix.png", dpi=150)
plt.show()
print("✅ Saved: confusion_matrix.png → commit this to your repo and include in README")

```



✓ **Saved:** confusion_matrix.png → commit this to your repo and include in README

```
# Print wrong predictions for your error analysis
# Review these carefully – pick 3 to analyze in depth in your README.

wrong_idx = np.where(ft_pred_ids != ft_true_ids)[0]
print(f"Wrong predictions: {len(wrong_idx)} / {len(ft_true_ids)}\n")

for i, idx in enumerate(wrong_idx[:15]):
    text = test_df.iloc[idx]["text"]
    true_label = ID_TO_LABEL[ft_true_ids[idx]]
    pred_label = ID_TO_LABEL[ft_pred_ids[idx]]
    confidence = ft_probs[idx][ft_pred_ids[idx]]
    print(f"--- #{i+1} ---")
    print(f"Text:      {text[:200]}{'...' if len(text) > 200 else ''}")
    print(f"True:       {true_label}")
    print(f"Predicted: {pred_label} (confidence: {confidence:.2f})")
    print()
```

ng predictions: 17 / 30

#1 ---

t: Because it's not really true. They didn't answer an ad. They were actual band for years prior to that.
e: analysis
dicted: hot_take (confidence: 0.35)

#2 ---

t: This is absolute nonsense. The metal fans hating on sleep token today never liked modern prog "metal"/djent in the first place
e: hot_take
dicted: analysis (confidence: 0.42)

#3 ---

t: OMG the number of times I watched the Wild Boys video drooling over JT. But their music was good. The stuff teenage girls line
e: anecdote
dicted: analysis (confidence: 0.38)

#4 ---

t: Honest answer? Teenage girls are mostly scared of ostracism and want to fit in with the crowd, this generally leads to having
e: hot_take
dicted: analysis (confidence: 0.42)

#5 ---

t: Misogyny. Look at how the Beatles were treated during their prime, and now they are considered the greatest 4 piece of all tin
e: hot_take
dicted: analysis (confidence: 0.41)

#6 ---

t: That's a strange conclusion to draw when I've repeatedly engaged with explanations that had nothing to do with misogyny.

fact, some of the comments I've appreciated most in this thread were the ...

e: anecdote
dicted: analysis (confidence: 0.40)

#7 ---

t: I'm gonna play devil's advocate and say it's not inherently sexism. In most cases the music that appeals to teenage girls is i
e: hot_take
dicted: analysis (confidence: 0.45)

#8 ---

t: The thing is, even if the music isn't juvenile and vapid, if it's popular with women, it's treated like it is.
e: hot_take
dicted: analysis (confidence: 0.37)

#9 ---

t: I think part of the issue here is there are at least 3 elements at play here: 1: the marketability element discussed above 2:
e: hot_take
dicted: analysis (confidence: 0.42)

#10 ---

```
t:      I wouldn't say I'm looking at it from the wrong angle, just a different one. I think what I was trying to get at is slightly c
e:      anecdote
dicted: analysis (confidence: 0.34)

#11 ---
t:      FKA Twigs stood out to me at Coachella this year in how dark and "weird" the performance was and I think this touches on what
e:      anecdote
```

✓ Section 5: Baseline Classifier (Groq)

Runs your zero-shot baseline using `llama-3.3-70b-versatile`.

You need to write the classification prompt using your label definitions.

```
from groq import Groq

# — TODO: Add your Groq API key —————
# Recommended: use Colab Secrets so your key is never visible in the notebook.
# 1. Click the 🔑 icon in the left sidebar ("Secrets")
# 2. Add a secret named GROQ_API_KEY with your key as the value
# 3. Enable notebook access for the secret
#
# Then uncomment Option A below (and delete Option B).
#
# Option A – Colab Secrets (recommended):
from google.colab import userdata
GROQ_API_KEY = userdata.get("GROQ_API_KEY")
#
# Option B – paste directly (do not commit to GitHub):
#GROQ_API_KEY = "your_groq_api_key_here"

assert GROQ_API_KEY, (
    "GROQ_API_KEY not set – add it in the Colab Secrets panel (\U0001f511, left "
    "sidebar) and enable notebook access for this notebook, or use Option B above."
)

client = Groq(api_key=GROQ_API_KEY)
print("✅ Groq client initialized")
```

✅ Groq client initialized

```
# — TODO: Write your classification prompt —————
# Your prompt should:
```

```
# 1. Name your community and task
# 2. Define each label in plain language (copy from your planning.md)
# 3. Give one example post per label
# 4. Tell the model to output ONLY the label name – nothing else
#
# The model's response must match one of your label strings exactly,
# or the classify_with_groq() function below will mark it as unparseable.
#
# _____
# REPLACE the placeholders below with your actual prompt. As written, this
# skeleton will NOT classify correctly – you must fill it in.
```

```
SYSTEM_PROMPT = """
```

```
You are classifying comments from r/LetsTalkMusic.
```

```
Assign each post to exactly one of the following categories.
```

```
analysis: The comment bypasses raw emotional framing to construct an objective sociological, historical, or industry-mechanic argument
Example: "That's because at one point, straight boys actually made up a material part of pop fandom. As they have moved into other mec
```

```
hot_take: A bold, highly confident assertion, ideological decree, or terminal dismissal stated as absolute fact without supporting ev:
Example: "Because patriarchy hates women. Don't worry about all the long-winded explanations."
```

```
anecdote: A first-person subjective narrative or nostalgic self-disclosure that engages with the discourse strictly through personal v
Example: "OMG the number of times I watched the Wild Boys video drooling over JT. But their music was good."
```

```
Respond with ONLY the label name.
```

```
Do not explain your reasoning.
```

```
Valid labels:
```

```
analysis
```

```
hot_take
```

```
anecdote
```

```
"""
```

```
print("Prompt length:", len(SYSTEM_PROMPT), "characters")
```

```
Prompt length: 1226 characters
```

```
def classify_with_groq(text):
    """Classify a single post. Returns a label string or None if unparseable."""
    try:
        response = client.chat.completions.create(
            model="llama-3.3-70b-versatile",
            messages=[
                {"role": "system", "content": SYSTEM_PROMPT},
```

```

        {"role": "user", "content": f"Classify this post:\n\n{text}"},
    ],
    temperature=0,
    max_tokens=20,
)
raw = response.choices[0].message.content.strip().lower()
# Match the model's output to a label. Check longest labels first so a
# label that is a substring of another (e.g. "recommendation" vs.
# "strong_recommendation") can't be matched by mistake.
for label in sorted(LABEL_MAP, key=len, reverse=True):
    if raw == label or label in raw:
        return label
return None # model output didn't match any known label
except Exception as e:
    print(f"API error: {e}")
    return None

# Run baseline on test set
print(f"Running baseline on {len(test_df)} examples...")
print("(May take a few minutes – 0.1s delay between requests to respect free-tier limits)\n")

baseline_preds = []
for i, (_, row) in enumerate(test_df.iterrows()):
    pred = classify_with_groq(row["text"])
    baseline_preds.append(pred)
    if (i + 1) % 10 == 0:
        print(f" {i+1}/{len(test_df)} complete...")
        time.sleep(0.1)

none_count = baseline_preds.count(None)
if none_count > 0:
    print(f"\n🚩 {none_count} responses could not be parsed.")
    print("Review your prompt – the model may not be outputting clean label names.")

```

Running baseline on 30 examples...
(May take a few minutes – 0.1s delay between requests to respect free-tier limits)

10/30 complete...
20/30 complete...
30/30 complete...

```

# Baseline metrics (exclude unparseable responses)
valid = [(p, t) for p, t in zip(baseline_preds, test_df["label_id"])
        if p is not None]
bl_pred_ids = [LABEL_MAP[p] for p, _ in valid]

```

```

bl_true_ids = [t for _, t in valid]

bl_accuracy = accuracy_score(bl_true_ids, bl_pred_ids)
print(f"🔗 Baseline accuracy: {bl_accuracy:.3f} "
      f"(evaluated on {len(valid)}/{len(test_df)} parseable responses)")
print()
label_names = [ID_TO_LABEL[i] for i in range(NUM_LABELS)]
print("Per-class metrics (baseline):")
print(classification_report(bl_true_ids, bl_pred_ids, target_names=label_names, zero_division=0))

```

🔗 Baseline accuracy: 0.900 (evaluated on 30/30 parseable responses)

Per-class metrics (baseline):

	precision	recall	f1-score	support
analysis	0.87	0.93	0.90	14
hot_take	0.90	0.82	0.86	11
anecdote	1.00	1.00	1.00	5
accuracy			0.90	30
macro avg	0.92	0.92	0.92	30
weighted avg	0.90	0.90	0.90	30

✓ Section 6: Compare Results and Export

Side-by-side comparison of both models.

Download the output files and commit them to your GitHub repo.

```

print("=" * 50)
print("RESULTS COMPARISON")
print("=" * 50)
print(f"{'Model':<35} {'Accuracy':>8}")
print("-" * 45)
print(f"{'Zero-shot baseline (Groq)':<35} {bl_accuracy:>8.3f}")
print(f"{'Fine-tuned DistilBERT':<35} {ft_accuracy:>8.3f}")
print("-" * 45)
delta = ft_accuracy - bl_accuracy
direction = "improvement" if delta >= 0 else "regression"
print(f"\nFine-tuning {direction}: {abs(delta):.3f}")
print()
print("Use these numbers in your README evaluation report.")

```

```
=====
RESULTS COMPARISON
=====
Model                                Accuracy
-----
Zero-shot baseline (Groq)            0.900
Fine-tuned DistilBERT                0.433
-----

Fine-tuning regression: 0.467

Use these numbers in your README evaluation report.
```

```
# Save results JSON – commit to your GitHub repo and reference in README
results = {
    "baseline_accuracy": round(bl_accuracy, 4),
    "finetuned_accuracy": round(ft_accuracy, 4),
    "improvement": round(ft_accuracy - bl_accuracy, 4),
    "test_set_size": len(test_df),
    "label_map": LABEL_MAP,
    "model": MODEL_NAME,
}
with open("evaluation_results.json", "w") as f:
    json.dump(results, f, indent=2)

print("✅ Files ready to download:")
print("  evaluation_results.json – metrics for your README")
print("  confusion_matrix.png    – include in your README")
print()
print("Download via: Files panel (📁) on the left → right-click → Download")
```

[Show hidden output](#)

```
from IPython.display import Image, display
display(Image(filename="confusion_matrix.png"))
```



```
-----  
FileNotFoundError                                Traceback (most recent call last)  
/tmp/ipykernel_1166/1282708405.py in <cell line: 0>()  
      1 from IPython.display import Image, display  
----> 2 display(Image(filename="confusion_matrix.png"))  
  
----- 3 frames -----  
/usr/local/lib/python3.12/dist-packages/IPython/core/display.py in reload(self)  
    660         """Reload the raw data from file or URL."""  
    661         if self.filename is not None:  
--> 662             with open(self.filename, self._read_flags) as f:  
    663                 self.data = f.read()  
    664             elif self.url is not None:  
  
FileNotFoundError: [Errno 2] No such file or directory: 'confusion_matrix.png'
```